

RAMAI

Codex Development Specification v1.0

AI Business Operating Assistant for PMI - Documento 11

Scopo. Trasformare la documentazione RAMAI in un processo di sviluppo agentico controllato, testabile e modulare, con regole precise su scope, repository, CI, review e sicurezza.

1. Scopo

Codex deve essere usato come software engineering agent dentro un processo controllato: specifiche -> issue -> branch -> code -> test -> review -> merge. Non deve inventare requisiti, cambiare architettura o ampliare lo scope senza una decisione documentata.

2. Repository structure

```
/apps/web # Next.js
/src/Ramai.Api # ASP.NET Core API/BFF
/src/Ramai.Worker # background jobs
/src/Ramai.Modules.* # bounded modules
/tests/unit
/tests/integration
/tests/architecture
/tests/e2e
/infra
/docs/adr
/docs/specs
```

3. Source of truth hierarchy

Priorità	Fonte
1	Security/privacy constraints e ADR approvati
2	Documenti 01-12 versionati
3	Issue/acceptance criteria dello sprint
4	Test automatici esistenti
5	Codice corrente
6	Assunzioni dell'agente - solo se non confliggono; devono essere dichiarate

4. Regole non negoziabili per Codex

- Non modificare schema DB senza migration e test.
- Non accedere direttamente alle tabelle di altri moduli.
- Non introdurre dipendenze nuove senza motivazione nella PR.
- Non inserire secret, token, credenziali o dati reali nel repository.
- Non disabilitare test per far passare CI.
- Non cambiare public API/contract senza aggiornare spec e contract test.
- Non implementare invio automatico email nell'MVP.

5. Issue template

Title: [MODULE] Outcome
Context: riferimento documento + sezione + user story
Goal: risultato utente/tecnico
In scope:
Out of scope:

Acceptance criteria:
Security/privacy constraints:
Telemetry required:
Tests required:
Definition of done:

6. Task sizing

Taglia	Regola
XS	1 file/local fix, < ~100 LOC, test semplice
S	1 componente o service, < ~300 LOC
M	feature verticale con API+domain+test; preferita massima per singolo task
L	deve essere spezzata prima di affidarla a Codex

7. Development workflow

Step	Output
1. Understand	sintesi requisiti e file interessati; nessuna implementazione prematura.
2. Plan	micro-plan con test e rischi.
3. Implement	minimo cambiamento coerente con architettura.
4. Test	unit/integration/architecture pertinenti.
5. Self-review	diff review, error paths, tenancy, logging, performance.
6. PR summary	cosa, perché, test, migration, risk, screenshots se UI.

8. Coding conventions

- C#: nullable enabled, async/await end-to-end, cancellation tokens, records/value objects dove appropriato.
- EF Core: query filtrate per TenantId, no raw SQL salvo review specifica.
- API: typed contracts, validation, ProblemDetails, version /api/v1.
- TypeScript: strict mode; runtime validation per payload esterni.
- Logging strutturato; no PII/body email nei log.
- Domain logic testabile senza framework/provider.

9. Architecture tests

CI deve bloccare violazioni dei confini: moduli non possono referenziare Infrastructure interna altrui; UI non accede al DB; provider adapters non diventano dipendenza del dominio.

10. Test requirements by change

Change	Minimum tests
Domain rule	unit
Repository/query	integration PostgreSQL + tenant isolation
API endpoint	integration/contract + authz
Connector mapping	contract replay + idempotency
AI prompt/schema	eval + schema validator
UI flow	component + e2e per critical path
Security-sensitive	negative tests + review manuale

11. CI pipeline

```
format/lint -> build -> unit -> architecture -> integration -> security scans  
-> AI eval smoke -> web tests -> e2e smoke -> container build
```

12. Branch/PR policy

- Feature branch per issue; PR piccole e reversibili.
- Merge solo con CI verde e review delle aree security/architecture quando toccate.
- Migration DB esplicite e backward-compatible quando possibile.
- Feature flag per comportamento rischioso o rollout graduale.

13. Codex prompt header standard

You are implementing RAMAI issue .
Use only the referenced specifications and repository conventions.
Do not broaden scope.
Protect tenant isolation and user data.
Before coding: summarize requirements, identify risks, list tests.
After coding: run relevant tests and report changed files, migrations, telemetry, residual risks.

14. Sprint 0 backlog

ID	Task
S0-01	Create monorepo structure + solution/workspaces
S0-02	Docker local stack PostgreSQL/Redis
S0-03	CI pipeline baseline
S0-04	Architecture test project
S0-05	Error handling/ProblemDetails conventions
S0-06	Logging/trace conventions
S0-07	ADR template + initial ADR-001..007
S0-08	Dev secrets and env configuration
S0-09	Testing fixtures/Testcontainers
S0-10	Code ownership and PR template

15. MVP epic order

Order	Epic
1	Identity + Tenant
2	Connector framework + Microsoft auth
3	Mail sync + normalization
4	Business Memory core
5	AI extraction pipeline
6	Insight Engine + Today
7	Ask RAMAI
8	Drafting + Briefing
9	Billing + entitlements
10	Hardening + beta instrumentation

16. Definition of Done

- Acceptance criteria soddisfatti e dimostrabili.
- Test pertinenti aggiunti e verdi.
- Nessuna violazione architecture/security scanning.
- Telemetry richiesta presente.
- Documentazione/API/schema aggiornati.
- Errore/retry/empty path gestiti.
- PR summary include residual risks.

17. Human gates

Anche se Codex produce l'intera implementazione, devono restare gate umani su: security model, database migrations distruttive, OAuth scopes, billing, privacy/deletion, release production e qualunque futura azione AI autonoma.

Stato documento: baseline v1.0. Le modifiche sostanziali devono essere versionate e tracciate.

Documento precedente: 10_Five_Year_Business_Plan_v1.0

Documento successivo previsto: 12_MVP_Test_Plan_v1.0

Versione: 1.0 - 29 settembre 2026